

Bloom — Technical Document

Version: 1.1

Author: AnastasiaE3

Last Updated: May 2026

1. Project Overview

Bloom is a personal calendar web app built with React on the frontend and Java Spring Boot on the backend. It allows the user to manage their schedule across multiple views (Year, Month, Week, Day), take notes, manage appointments, and keep personal lists.

In Phase 2, the app will connect to Gmail and Android SMS to automatically detect and import appointment confirmations.

2. Tech Stack

Frontend

Technology	Purpose
React	Frontend UI framework
React Router	Navigation between views
Tailwind CSS	Styling and layout
localStorage	Temporary data storage (Phase 1)

Backend (Phase 2)

Technology	Purpose
Java	Primary backend language
Spring Boot	REST API framework
Spring Security	Authentication and authorization
Google OAuth 2.0	Gmail login and access

Technology	Purpose
Gmail API	Read and parse appointment emails
PostgreSQL	Primary database
JPA / Hibernate	Database ORM (maps Java objects to DB tables)

DevOps & Tools

Technology	Purpose
GitHub	Version control and code hosting
GitHub Projects	Kanban board for task management
Postman	API testing

3. Database Design (Phase 2)

PostgreSQL Tables

users

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique user ID
email	VARCHAR	User's email address
name	VARCHAR	User's display name
google_token	TEXT	OAuth token for Gmail access
created_at	TIMESTAMP	Account creation date

events

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique event ID
user_id	INTEGER	Foreign key to users
title	VARCHAR	Event title
date	DATE	Event date
start_time	TIME	Start time
end_time	TIME	End time
location	VARCHAR	Event location
notes	TEXT	Additional notes
source	VARCHAR	manual / gmail / sms
created_at	TIMESTAMP	Creation timestamp

notes

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique note ID
user_id	INTEGER	Foreign key to users
date	DATE	The day this note belongs to
content	TEXT	Note content
updated_at	TIMESTAMP	Last updated timestamp

lists

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique list ID
user_id	INTEGER	Foreign key to users
date	DATE	The day this list belongs to
name	VARCHAR	List name (e.g. Grocery)
created_at	TIMESTAMP	Creation timestamp

list_items

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique item ID
list_id	INTEGER	Foreign key to lists
content	VARCHAR	Item text
is_checked	BOOLEAN	Whether item is checked off

reminders

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique reminder ID
event_id	INTEGER	Foreign key to events
remind_at	TIMESTAMP	When to send the reminder
type	VARCHAR	week_before / day_before / 30_min_before
is_sent	BOOLEAN	Whether reminder was sent

4. API Endpoints (Phase 2)

Events

Method	Endpoint	Description
GET	<code>/api/events?date=2026-05-06</code>	Get all events for a date
POST	<code>/api/events</code>	Create a new event
PUT	<code>/api/events/{id}</code>	Update an event
DELETE	<code>/api/events/{id}</code>	Delete an event

Notes

Method	Endpoint	Description
GET	<code>/api/notes?date=2026-05-06</code>	Get note for a date
POST	<code>/api/notes</code>	Create or update a note

Lists

Method	Endpoint	Description
GET	<code>/api/lists?date=2026-05-06</code>	Get all lists for a date
POST	<code>/api/lists</code>	Create a new list
POST	<code>/api/lists/{id}/items</code>	Add item to a list
PUT	<code>/api/lists/items/{id}</code>	Toggle checkbox on item
DELETE	<code>/api/lists/{id}</code>	Delete a list

Gmail Integration

Method	Endpoint	Description
GET	<code>/api/gmail/sync</code>	Scan Gmail for appointment confirmations
POST	<code>/api/gmail/import/{emailId}</code>	Import a specific email as an event

5. Gmail & SMS Integration (Phase 2)

Gmail

- User logs in with Google OAuth 2.0
- Spring Boot backend requests read-only Gmail access
- Gmail API scans inbox for keywords: `confirmation`, `appointment`, `booking`, `reservation`, `scheduled`
- Detected emails are parsed for date, time, and location
- Parsed data is automatically added to the calendar as an event
- User gets a notification: *"New appointment found in your Gmail — added to your calendar!"*

Android SMS

- User grants SMS read permission
 - App scans recent messages for appointment-related keywords
 - Parsed SMS data is imported as calendar events
 - Same notification flow as Gmail
-

6. Reminders & Notifications

Reminder	Timing
Early reminder	1 week before
Day reminder	1 day before
Final reminder	30 minutes before

- **Phase 1:** Reminder badges displayed visually in the app
 - **Phase 2:** Browser push notifications via Web Push API
 - Backend checks for upcoming reminders every 15 minutes using a scheduled Spring `@Scheduled` job
-

7. Project Structure

```
my-calendar-app/
├── frontend/                                (React)
│   ├── public/
│   └── src/
│       ├── views/
│       │   ├── YearView.js
│       │   ├── MonthView.js
│       │   ├── WeekView.js
│       │   └── DayView.js
│       ├── components/
│       │   ├── Header.js
│       │   ├── AppointmentCard.js
│       │   ├── NoteSection.js
│       │   └── CheckList.js
│       ├── App.js
│       ├── index.js
│       └── index.css
├── backend/                                (Java Spring Boot)
│   └── src/main/java/
│       ├── controllers/
│       │   ├── EventController.java
│       │   ├── NoteController.java
│       │   ├── ListController.java
│       │   └── GmailController.java
│       ├── services/
│       │   ├── EventService.java
│       │   ├── GmailService.java
│       │   └── ReminderService.java
│       ├── models/
│       │   ├── Event.java
│       │   ├── Note.java
│       │   ├── List.java
│       │   └── User.java
│       └── repositories/
│           ├── EventRepository.java
│           └── NoteRepository.java
└── TECHNICAL.md
```

8. Development Phases

✅ Phase 1 — Frontend (Current)

- React app with all 4 views
- localStorage for data persistence
- Full UI with colors, navigation, day sections

➡ SOON Phase 2 — Backend

- Java Spring Boot REST API
 - PostgreSQL database
 - Gmail OAuth integration
 - SMS parsing
 - Browser push notifications
-

9. Kanban Board Tasks

🔧 Setup

- ☒ Set up React app with create-react-app
- ☒ Connect local project to GitHub repo
- ☒ Install dependencies (Tailwind CSS, React Router)

🎨 Design & Layout

- ☒ Set up color theme and global styles
- ☒ Build the main app layout (header, navigation)
- ☐ Build Year view
- ☐ Build Month view
- ☐ Build Week view
- ☐ Build Day view

Features

- ☐ Add appointments & meetings section
- ☐ Add notes section
- ☐ Add lists with checkboxes section
- ☐ Add reminder badges to appointments
- ☐ Click a day → navigate to Day view

Data

- ☐ Save all data to localStorage
- ☐ Load saved data on app start

Phase 2 — Backend

- ☐ Set up Java Spring Boot project
- ☐ Connect PostgreSQL database
- ☐ Build REST API endpoints
- ☐ Gmail OAuth integration
- ☐ SMS parsing
- ☐ Browser push notifications

Built with  by AnastasiaE3